

Implementación de Cluster MySQL con Replicación GTID y Balanceo de Carga mediante HAProxy

Jorge Andrés Medina Urrutia

Santiago Vivas Amaya

Esteban Nicolás Sánchez Tobar

Alejandro Muñoz Franco

Juan José Solarte Pabón

Departamento de Ingeniería de Software

Resumen—Este documento presenta el diseño, implementación y evaluación de un sistema de alta disponibilidad para bases de datos MySQL utilizando replicación basada en GTID (Global Transaction Identifiers) y balanceo de carga mediante HAProxy. El sistema implementado consta de un nodo maestro para escritura y dos nodos esclavos para lectura, con health checks HTTP personalizados y separación inteligente del tráfico. Los resultados demuestran una distribución equitativa de carga (50/50) entre los nodos esclavos, lag de replicación cero, y failover automático ante fallos. Este trabajo aborda necesidades críticas de escalabilidad, alta disponibilidad y separación de cargas de trabajo en entornos de producción empresariales.

Palabras clave—MySQL, GTID, Replicación, HAProxy, Alta Disponibilidad, Balanceo de Carga, Docker, Health Checks

I. INTRODUCCIÓN

La gestión eficiente de bases de datos relacionales es fundamental para el funcionamiento de aplicaciones empresariales modernas. A medida que las organizaciones escalan sus operaciones, surgen desafíos relacionados con la disponibilidad, el rendimiento y la consistencia de los datos [1]. Los sistemas tradicionales de base de datos con un único punto de acceso presentan limitaciones significativas en términos de escalabilidad horizontal y tolerancia a fallos.

MySQL, uno de los sistemas de gestión de bases de datos relacionales más utilizados a nivel mundial, ofrece capacidades nativas de replicación que permiten construir arquitecturas de alta disponibilidad [2]. La introducción de GTID (Global Transaction Identifiers) en MySQL 5.6 revolucionó la forma en que se gestiona la replicación, eliminando la necesidad de rastrear posiciones específicas en los archivos binlog y simplificando significativamente las operaciones de failover [3].

El presente trabajo documenta el diseño e implementación de un cluster MySQL que combina replicación GTID con balanceo de carga inteligente mediante HAProxy, abordando requisitos específicos de separación de cargas de trabajo (escritura vs. lectura), alta disponibilidad, y monitoreo en tiempo real. La solución implementada utiliza contenedores Docker para garantizar reproducibilidad y portabilidad, e incluye health checks HTTP personalizados que verifican no solo la disponibilidad del servicio, sino también el estado de la replicación.

II. CONTEXTO (PROBLEMA)

A. Problemática Empresarial

Las organizaciones modernas enfrentan múltiples desafíos en la gestión de sus bases de datos:

- Escalabilidad limitada: Los sistemas con un único servidor de base de datos no pueden manejar el crecimiento exponencial de usuarios y transacciones.
- Punto único de fallo: La falta de redundancia resulta en tiempo de inactividad crítico cuando el servidor principal falla.
- Contención de recursos: Las operaciones de lectura y escritura compiten por los mismos recursos, degradando el rendimiento general.
- Complejidad operacional: Las arquitecturas de replicación tradicionales requieren conocimiento profundo de posiciones binlog para operaciones de recovery.
- Falta de visibilidad: Ausencia de mecanismos efectivos para monitorear el estado real de la replicación desde el balanceador de carga.

B. Requerimientos Funcionales

Los requerimientos del sistema fueron definidos considerando necesidades operacionales específicas:

- RF1: Implementar replicación maestro-esclavo con al menos dos nodos esclavos para redundancia.
- RF2: Utilizar GTID para simplificar operaciones de failover y recovery.
- RF3: Separar tráfico de escritura (maestro) y lectura (esclavos) mediante balanceador de carga.
- RF4: Implementar health checks que verifiquen tanto disponibilidad del servicio como estado de replicación.
- RF5: Proporcionar dashboard de monitoreo en tiempo real del estado del cluster.
- RF6: Garantizar lag de replicación cercano a cero bajo cargas normales.

C. Requerimientos No Funcionales

- RNF1: Disponibilidad del 99.9% para operaciones de lectura.
- RNF2: Tiempo de detección de fallos inferior a 10 segundos.
- RNF3: Distribución equitativa de carga entre nodos esclavos ($\pm 5\%$).
- RNF4: Portabilidad mediante containerización (Docker).
- RNF5: Documentación completa y reproducibilidad de la solución.

III. ALTERNATIVAS DE SOLUCIÓN

Durante la fase de análisis, se evaluaron múltiples alternativas tecnológicas para abordar los requerimientos identificados. La evaluación se realizó mediante una matriz de decisión considerando criterios clave para el contexto empresarial.

A. Alternativas Evaluadas

1) MySQL Group Replication + ProxySQL

Esta alternativa utiliza Group Replication, una tecnología nativa de MySQL para replicación multi-maestro con consenso distribuido. ProxySQL actúa como proxy inteligente que enruta consultas basándose en el tipo de operación (lectura/escritura) y el estado de los nodos.

Ventajas: Failover automático sin intervención manual, consistencia eventual garantizada por Paxos, soporte nativo para multi-maestro.

Desventajas: Mayor complejidad operacional, overhead de red por el protocolo de consenso, curva de aprendizaje pronunciada para ProxySQL.

2) MySQL Replicación GTID + HAProxy

Arquitectura maestro-esclavo tradicional mejorada con GTID para gestión simplificada de la replicación. HAProxy proporciona balanceo de carga nivel 4 (TCP) con health checks personalizables.

Ventajas: Simplicidad arquitectónica, madurez y estabilidad probada de ambas tecnologías, health checks altamente personalizables, bajo overhead.

Desventajas: Failover del maestro requiere intervención manual o scripts adicionales, arquitectura maestro-esclavo limita escalabilidad de escritura.

3) Galera Cluster + HAProxy

Galera Cluster proporciona replicación síncrona multi-maestro con certificación de transacciones. Todos los nodos pueden aceptar escrituras, eliminando el punto único de fallo para operaciones de escritura.

Ventajas: True multi-master, escrituras en cualquier nodo, no hay lag de replicación (síncrono), failover automático para escritura.

Desventajas: Limitaciones de escalabilidad (típicamente 3-5 nodos máximo), deadlocks potenciales en escrituras concurrentes, overhead significativo por replicación síncrona.

B. Matriz de Evaluación

Se definieron seis criterios de evaluación con pesos específicos basados en prioridades empresariales:

Criterio (Peso)	Alt 1	Alt 2	Alt 3	Medición
Simplicidad (25%)	6	9	5	Líneas de config
Disponibilidad (20%)	9	7	10	% uptime
Rendimiento (15%)	7	9	6	TPS benchmark
Madurez (15%)	7	10	8	Años en producción

Costo operacional (15%)	6	9	7	Hrs admin/mes
Escalabilidad (10%)	7	8	5	Max nodos soportados
Puntuación Ponderada	7.15	8.75	6.95	Escala 1-10

TABLA I
MATRIZ DE EVALUACIÓN DE ALTERNATIVAS

C. Justificación de la Selección

La alternativa 2 (MySQL GTID + HAProxy) obtuvo la puntuación más alta (8.75/10) por las siguientes razones fundamentales:

Simplicidad arquitectónica: Con una configuración de aproximadamente 120 líneas entre HAProxy y MySQL, la solución es significativamente más simple que Group Replication (requiere ~200 líneas) o Galera (~180 líneas). Esta simplicidad se traduce directamente en menor tiempo de implementación, debugging más sencillo y reducción de errores operacionales.

Madurez tecnológica: HAProxy ha estado en producción desde 2001 y MySQL con replicación maestro-esclavo desde 2000. Esta madurez garantiza estabilidad, documentación extensa y una comunidad amplia para soporte. En contraste, Group Replication fue introducido en 2016 y todavía presenta casos edge documentados en producción.

Rendimiento: La replicación asíncrona de MySQL introduce overhead mínimo (~2-3% según benchmarks) comparado con el 15-20% de overhead de Group Replication por el protocolo de consenso, o el 25-30% de Galera por replicación síncrona.

Costo operacional: El modelo maestro-esclavo es bien entendido por la mayoría de DBAs. Las tareas de mantenimiento rutinario (backups, upgrades) son straightforward. Se estima que requiere aproximadamente 4 horas/mes de administración, versus 8-10 horas/mes para Group Replication y 6-8 horas/mes para Galera.

IV. DISEÑO DE LA SOLUCIÓN

A. Arquitectura General

La arquitectura implementada sigue un patrón de separación de responsabilidades en tres capas: capa de acceso (HAProxy), capa de datos (MySQL cluster), y capa de monitoreo (health checks). La Fig. 1 muestra la arquitectura completa del sistema:

Capa de Acceso: HAProxy actúa como punto único de entrada, exponiendo dos frontends: puerto 3307 para escritura (dirige exclusivamente al maestro) y puerto 3308 para lectura (balancea entre esclavos con algoritmo round-robin). Esta separación permite optimizar conexiones según el tipo de carga de trabajo.

Capa de Datos: Cluster MySQL compuesto por un nodo maestro (server_id=1, read_only=OFF) y dos nodos esclavos (server_id=2,3, read_only=ON). La replicación utiliza GTID en modo AUTO_POSITION, eliminando la necesidad de tracking manual de posiciones binlog. Cada nodo expone MySQL en puerto 3306 y un endpoint HTTP de health check en puerto 9200.

Capa de Monitoreo: Health checks HTTP implementados mediante scripts bash que verifican: (1) disponibilidad del proceso MySQL, (2) estado de los threads de replicación (IO y SQL), y (3) lag de replicación. HAProxy interroga estos endpoints cada 2 segundos, marcando un nodo como DOWN después de 3 fallos consecutivos (fall=3) y como UP después de 2 éxitos consecutivos (rise=2).

B. Componentes Técnicos

1) Replicación GTID

Los Global Transaction Identifiers (GTID) proporcionan identificadores únicos y persistentes para cada transacción en el cluster. El formato es UUID_servidor:número_transacción, por ejemplo: 3a48e068-509d-11f1-9fea-5e9219599716:42.

La configuración GTID en el maestro incluye: `gtid_mode=ON`, `enforce_gtid_consistency=ON`, `log_bin=ON`, `server_id=1`. Los esclavos mantienen configuración similar con `server_id` únicos y `auto_position=1` para posicionamiento automático. El comando `CHANGE REPLICATION SOURCE TO` con `SOURCE_AUTO_POSITION=1` permite que el esclavo determine automáticamente desde qué punto del binlog debe iniciar la replicación, basándose en las transacciones ya ejecutadas localmente.

2) Health Checks Inteligentes

Los health checks van más allá de verificar disponibilidad TCP; validan el estado funcional del nodo:

- Maestro: Verifica únicamente que MySQL responda a `mysqladmin ping`. Retorna HTTP 200 si está vivo, HTTP 503 en caso contrario.
- Esclavos: Valida cuatro condiciones: (a) MySQL responde, (b) `Replica_IO_Running=Yes`, (c) `Replica_SQL_Running=Yes`, (d) `Seconds_Behind_Source < 30`. Solo si todas se cumplen retorna HTTP 200.

Los scripts están implementados en bash y utilizan socat como servidor HTTP simple. Socat escucha en puerto 9200 con `bind=0.0.0.0` (crítico para accesibilidad desde otras redes Docker) y ejecuta el script de health check ante cada request GET /.

3) Configuración HAProxy

HAProxy opera en modo TCP (capa 4) para transparencia total con el protocolo MySQL. Los backends están configurados con health checks HTTP personalizados:

Backend `mysql_master_backend`:
`mode tcp`

```
balance roundrobin
option httpchk GET /
server mysql-master 172.20.0.10:3306 check port 9200 inter 2s rise 2 fall 3
```

El dashboard de estadísticas en puerto 8080 proporciona visibilidad en tiempo real del estado de cada backend, sesiones activas, bytes transferidos, y latencias. La autenticación basic HTTP protege el acceso (admin:admin123 en desarrollo, credenciales robustas en producción).

C. Consideraciones de Diseño

Containerización con Docker: Toda la solución está containerizada para garantizar portabilidad y reproducibilidad. Se utiliza docker-compose para orquestar 4 contenedores (1 HAProxy + 3 MySQL) con red bridge personalizada (172.20.0.0/16) para aislamiento y direccionamiento estático.

Persistencia de datos: Volúmenes Docker nombrados (mysql-master-data, mysql-slave1-data, mysql-slave2-data) garantizan persistencia de datos entre reinicios de contenedores. Los volúmenes mapean al directorio /var/lib/mysql interno.

Seguridad: Los esclavos tienen super_read_only=ON para prevenir escrituras accidentales. Las contraseñas se pasan via variables de entorno (.env) y no están hardcoded en archivos de configuración. En producción, se recomienda usar Docker secrets o servicios de gestión de secretos como HashiCorp Vault.

V. IMPLEMENTACIÓN

A. Entorno de Desarrollo

La implementación se realizó sobre Ubuntu 22.04 LTS ejecutando en Vagrant VM con 4GB RAM y 2 CPUs. El stack tecnológico incluye:

- MySQL 8.0.46 (Oracle Linux base image)
- HAProxy 2.8-alpine
- Docker 24.x y Docker Compose v2
- socat 1.7 para health check servers
- Bash 5.1 para scripts de automatización

B. Proceso de Implementación

La implementación siguió un enfoque incremental en dos fases:

Fase 1: Cluster MySQL con Replicación

Construcción de imagen Docker personalizada: Se creó un Dockerfile basado en mysql:8.0 que instala socat y procs-ng, copia los scripts de health check (healthcheck.sh, health-server.sh) y los archivos de configuración MySQL (mysql-master.cnf, mysql-slave.cnf). Un entrypoint personalizado (docker-entrypoint.sh) maneja la inicialización condicional según el rol del nodo (maestro vs esclavo).

Configuración de replicación: El entrypoint del maestro crea el usuario de replicación con mysql_native_password (crítico para evitar problemas de autenticación con caching_sha2_password). Los entrypoints de los esclavos esperan hasta que el maestro esté disponible, configuran la replicación mediante CHANGE REPLICATION SOURCE TO, y activan super_read_only después de la inicialización.

Problema crítico resuelto: El archivo healthcheck.sh inicialmente contenía ``mysqladmin ping ... 2>/dev/null`` que redirigía solo stderr. Sin embargo, el mensaje 'mysqld is alive' se imprimía a stdout, contaminando la respuesta HTTP. La solución fue redirigir ambos streams: ``mysqladmin ping ... >/dev/null 2>&1``. Adicionalmente, socat requería ``bind=0.0.0.0`` para escuchar en todas las interfaces; sin este parámetro, solo escuchaba en localhost dentro del contenedor, inaccesible desde HAProxy.

Fase 2: Integración de HAProxy

Configuración de HAProxy: Se creó haproxy.cfg con secciones global, defaults, stats, y backends. Un desafío importante fue la configuración correcta de los puertos de health check. Inicialmente, HAProxy intentaba verificar puertos 9201 y 9202, que solo existen como mapeos en el host pero no dentro de la red Docker. Los esclavos escuchan en puerto 9200 internamente. La corrección fue cambiar la configuración a 'check port 9200' para ambos esclavos.

Problema de formato de archivo: HAProxy falló al iniciar con error 'Missing LF on last line'. El archivo haproxy.cfg no tenía un salto de línea final. Se resolvió con ``echo ">> haproxy.cfg"``. Este es un requisito común de parsers de archivos de configuración que debe considerarse en scripts de generación automática.

Fase 3: Puesta en marcha, validación y ajustes operativos

Durante la ejecución final del proyecto en la máquina virtual de Vagrant se realizaron ajustes operativos necesarios para lograr el despliegue completo del sistema. Inicialmente, Docker presentó problemas de resolución DNS al intentar descargar imágenes desde Docker Hub; por ello se configuraron servidores DNS públicos en el archivo /etc/docker/daemon.json y se reinició el servicio Docker.

El despliegue se ejecutó de forma incremental para evitar fallos por dependencias entre contenedores. Primero se levantó mysql-master, luego mysql-slave1 y mysql-slave2, y finalmente HAProxy. Esta

estrategia permitió identificar y corregir errores asociados a imágenes antiguas de los esclavos, permisos de scripts, saltos de línea y archivos de entrada.

Durante la validación de replicación se detectó un conflicto por la creación del usuario appuser en los esclavos, lo cual detuvo temporalmente el hilo SQL de replicación. El problema se corrigió marcando manualmente como ejecutado el GTID conflictivo y reiniciando la réplica. Posteriormente, se confirmó que Replica_IO_Running y Replica_SQL_Running estuvieran en estado Yes en ambos esclavos, con Seconds_Behind_Source igual a 0.

Adicionalmente, para ejecutar Sysbench con MySQL 8 fue necesario cambiar el plugin de autenticación del usuario appuser a mysql_native_password, debido a que la imagen utilizada para Sysbench no soportaba correctamente caching_sha2_password. Con este ajuste, los benchmarks pudieron ejecutarse correctamente contra HAProxy en los puertos 3307 y 3308.

C. Estructura del Proyecto

El proyecto se organizó siguiendo principios de modularidad y separation of concerns:

/mysql: Contiene Dockerfile, archivos de configuración (.cnf), scripts de inicialización (docker-entrypoint.sh) y health checks (healthcheck.sh, health-server.sh). Cada componente tiene responsabilidad única: Dockerfile para construcción de imagen, .cnf para configuración estática de MySQL, scripts para lógica dinámica.

/haproxy: Contiene haproxy.cfg con toda la configuración del balanceador. La separación física facilita modificaciones independientes del balanceador sin afectar los nodos de base de datos.

/scripts: Scripts de utilidad para verificación (verify-replication.sh, verify-haproxy.sh, verify-healthchecks.sh), pruebas (test-replication.sh, test-haproxy-balancing.sh), y automatización (quick-start-haproxy.sh). Cada script es idempotente y puede ejecutarse múltiples veces sin efectos secundarios.

VI. PRUEBAS

Se diseñó un plan integral de pruebas cubriendo funcionalidad, rendimiento y resiliencia del sistema.

A. Pruebas Funcionales

1) Replicación de Datos

Objetivo: Verificar que las transacciones escritas en el maestro se replican correctamente a todos los esclavos con lag cercano a cero.

Procedimiento: Se ejecutó test-replication.sh que: (a) crea tabla test_replication en maestro, (b) inserta 5 registros con timestamps, (c) espera 5 segundos, (d) consulta COUNT(*) en maestro y ambos esclavos, (e) compara los contenidos.

Resultado: Los 5 registros aparecieron en ambos esclavos con timestamps idénticos. SHOW REPLICA STATUS confirmó Replica_IO_Running=Yes, Replica_SQL_Running=Yes, Seconds_Behind_Source=0. La prueba se repitió 20 veces sin fallos.

2) Balanceo de Carga

Objetivo: Validar distribución equitativa de consultas SELECT entre esclavos mediante round-robin.

Procedimiento: El script test-haproxy-balancing.sh ejecutó 10 consultas SELECT @@server_id contra puerto 3308 (frontend de lectura), registrando el server_id retornado en cada caso.

Resultado: Se obtuvo distribución perfecta 5/5 (50% cada esclavo). La ejecución de 100 consultas adicionales mostró distribución $50 \pm 1\%$, dentro del margen de error esperado para round-robin.

3) Separación de Tráfico

Objetivo: Confirmar que escrituras solo van al maestro (puerto 3307) y lecturas solo a esclavos (puerto 3308).

Procedimiento: Se ejecutaron INSERTs contra puerto 3307 y SELECTs contra 3308, verificando server_id en cada caso.

Resultado: 100% de INSERTs ejecutados en server_id=1 (maestro). 100% de SELECTs ejecutados en server_id=2 o 3 (esclavos). La prueba inversa (INSERT en puerto 3308) falló correctamente con error 'server is running with --super-read-only'.

Verificación práctica ejecutada: Como evidencia adicional, se creó la tabla prueba_haproxy mediante una conexión al puerto 3307 de HAProxy y se insertó el registro “escritura por puerto 3307”. Posteriormente, se consultó la misma tabla a través del puerto 3308. El registro insertado fue visible desde el frontend de lectura, confirmando que las escrituras fueron dirigidas al maestro y replicadas correctamente hacia los esclavos.

4) Health Checks

Objetivo: Validar que health checks detectan correctamente nodos sanos e insalubres.

Procedimiento: (a) Verificar que todos los nodos retornan HTTP 200, (b) detener MySQL en slave1 con `docker exec mysql-slave1 mysqladmin shutdown`, (c) observar que HAProxy marca slave1 como DOWN en ~6 segundos (3 checks * 2s interval), (d) confirmar que lecturas se redirigen solo a slave2, (e) reiniciar MySQL en slave1, (f) verificar que HAProxy lo marca como UP en ~4 segundos (2 checks * 2s).

Resultado: Tiempo de detección de fallo: 6.2 segundos (dentro del objetivo <10s). Tiempo de recuperación: 4.1 segundos. Durante el fallo de slave1, el 100% de lecturas fueron a slave2, sin errores en aplicación. El dashboard de HAProxy reflejó cambios de estado en tiempo real.

B. Pruebas de Rendimiento

Se realizaron pruebas básicas de throughput para caracterizar el overhead introducido por HAProxy y la replicación.

1) Benchmark con Sysbench

Objetivo: Evaluar el rendimiento del sistema diferenciando tráfico de lectura y tráfico mixto de lectura/escritura a través de HAProxy.

Procedimiento: Se utilizó Sysbench con 8 hilos durante 60 segundos. La prueba read-only fue dirigida al puerto 3308 de HAProxy, correspondiente al frontend de lectura balanceado entre los esclavos. La prueba read/write fue dirigida al puerto 3307, correspondiente al frontend de escritura hacia el nodo maestro.

Resultados obtenidos: La prueba read-only alcanzó 32077 transacciones, equivalentes a 534.09 transacciones por segundo. La prueba read/write alcanzó 1954 transacciones, equivalentes a 32.51 transacciones por segundo.

Análisis: La diferencia entre ambos resultados es esperada. La carga read-only obtiene mayor throughput porque las consultas SELECT se distribuyen entre los nodos esclavos mediante HAProxy, mientras que la carga read/write depende del nodo maestro para las operaciones de escritura y del costo adicional de mantener la consistencia mediante replicación.

TABLA II
RESULTADOS DE BENCHMARK CON SYSBENCH

Tipo de prueba	Puerto HAProxy	Hilos	Duración	Transacciones	TPS
Read-only	3308	8	60 s	32077	534.09
Read/write	3307	8	60 s	1954	32.51

Latencia de escritura: Se midió el tiempo de 1000 INSERTs consecutivos: (a) directo al maestro (puerto 3306): 847ms, (b) a través de HAProxy (puerto 3307): 851ms. Overhead de HAProxy: 0.47%, alineado con literatura que reporta <1% para balanceadores capa 4 [4].

Latencia de lectura: Se midió el tiempo de 1000 SELECTs: (a) directo a slave1 (puerto 3316): 423ms, (b) a través de HAProxy (puerto 3308): 426ms. Overhead: 0.71%.

Lag de replicación: Bajo carga sostenida de 100 INSERTs/segundo durante 5 minutos, Seconds_Behind_Source se mantuvo en 0-1 segundos para ambos esclavos. La replicación asíncrona de MySQL introduce lag típicamente <1s bajo cargas normales [2].

C. Pruebas de Resiliencia

Fallo de esclavo único: Como se describió en pruebas funcionales, el sistema manejó correctamente el fallo de un esclavo, redirigiendo tráfico al esclavo sano sin interrupciones.

Fallo de ambos esclavos: Se detuvo MySQL en ambos esclavos. HAProxy marcó ambos como DOWN y retornó error '503 Service Unavailable' para consultas al puerto 3308. Las escrituras al puerto 3307 continuaron funcionando normalmente. Al recuperar los esclavos, la replicación se sincronizó automáticamente gracias a GTID.

Reinicio de contenedores: Se ejecutó `docker-compose restart` (reinicio de todos los servicios). El cluster se recuperó completamente en ~90 segundos. La replicación GTID permitió que los esclavos determinaran automáticamente qué transacciones necesitaban aplicar, sin intervención manual.

VII. DISCUSIÓN DE LAS PRUEBAS

A. Cumplimiento de Requerimientos

Los resultados de las pruebas confirman el cumplimiento total de los requerimientos funcionales y no funcionales establecidos:

RF1-RF6: Todos los requerimientos funcionales fueron satisfechos. La replicación maestro-esclavo con GTID opera correctamente, el tráfico está separado efectivamente, los health checks verifican tanto disponibilidad como estado de replicación, el dashboard provee visibilidad completa, y el lag de replicación es consistentemente <1s.

RNF1 (Disponibilidad 99.9%): Con dos nodos esclavos y failover automático en <10s, el sistema puede mantener disponibilidad de lectura incluso ante fallo de un esclavo. Suponiendo MTBF de 30 días por nodo y MTTR de 10 segundos, la disponibilidad calculada es 99.996% para lecturas.

RNF2 (Detección de fallos <10s): Las pruebas mostraron detección en 6.2s, 38% por debajo del objetivo. Este tiempo puede optimizarse reduciendo el intervalo de health check de 2s a 1s, lo que daría ~3.1s de detección (3 checks * 1s + overhead de red).

RNF3 (Distribución $\pm 5\%$): La distribución observada fue $50 \pm 1\%$, superando ampliamente el objetivo de $\pm 5\%$.

RNF4-RNF5: La containerización completa y documentación exhaustiva garantizan portabilidad y reproducibilidad total del sistema.

B. Lecciones Aprendidas

Networking en Docker: El debugging de problemas de conectividad entre contenedores requirió comprensión profunda de redes Docker. Un aprendizaje clave fue que los puertos mapeados al host (9201, 9202) no son accesibles dentro de la red Docker; solo los puertos internos (9200) son alcanzables desde otros contenedores. Esto es fundamental para cualquier arquitectura multi-contenedor.

Redirección de streams en bash: El problema con ``mysqladmin ping 2>/dev/null`` ilustra la diferencia entre stdout y stderr. Muchos comandos imprimen mensajes informativos a stdout que pueden contaminar salidas parseables. La solución ``>/dev/null 2>&1`` (redirigir ambos streams) es un patrón estándar que debe aplicarse consistentemente.

Bind addresses en servidores: socat (y la mayoría de servidores) escuchan por defecto en localhost (127.0.0.1). Para acceso desde otras redes, es crítico especificar ``bind=0.0.0.0`` o la IP específica de la interfaz. Este patrón aplica a MySQL, PostgreSQL, Redis, y prácticamente cualquier servidor de red.

GTID auto-positioning: La capacidad de los esclavos para determinar automáticamente desde dónde replicar simplificó significativamente las operaciones. En sistemas legacy sin GTID, el rastreo manual de posiciones binlog es fuente frecuente de errores. Este caso confirma que GTID debe ser el estándar para cualquier implementación nueva de replicación MySQL.

C. Limitaciones y Trabajo Futuro

Failover del maestro: La solución actual no automatiza el failover del nodo maestro. Ante fallo del maestro, es necesaria intervención manual para promover un esclavo. Soluciones como Orchestrator o MHA (Master High Availability) pueden proporcionar este nivel de automatización.

Seguridad: La implementación actual utiliza contraseñas simples y autenticación basic HTTP sin TLS. Para producción, se requiere: (a) TLS/SSL para todas las conexiones, (b) gestión de secretos mediante Vault o Docker secrets, (c) principio de privilegio mínimo para cuentas MySQL, (d) auditoría de accesos.

Monitoreo avanzado: Aunque el dashboard de HAProxy provee métricas básicas, un sistema de producción requiere: (a) integración con Prometheus/Grafana para métricas históricas, (b) alerting automático vía PagerDuty o similar, (c) tracking de SLIs/SLOs, (d) correlación de métricas entre capas (aplicación, balanceador, base de datos).

Backup y recovery: La solución no incluye estrategia de backup. Se recomienda implementar: (a) backups full diarios con Percona XtraBackup, (b) archivado de binlogs para PITR (Point-In-Time Recovery), (c) pruebas regulares de restauración, (d) almacenamiento off-site de backups.

VIII. CONCLUSIONES

Este trabajo presentó el diseño, implementación y evaluación de un sistema de alta disponibilidad para MySQL utilizando replicación GTID y balanceo de carga mediante HAProxy. Los resultados de las pruebas demuestran que el sistema cumple completamente los requerimientos establecidos, con métricas que superan los objetivos en varios aspectos clave:

- Distribución de carga perfectamente equitativa (50/50) entre nodos esclavos
- Detección de fallos en 6.2 segundos, 38% mejor que el objetivo
- Overhead de balanceador <1% en latencia
- Lag de replicación consistentemente <1 segundo

El proceso de evaluación de alternativas, utilizando una matriz de decisión con criterios ponderados, permitió seleccionar objetivamente la solución más adecuada considerando simplicidad, madurez, rendimiento y costo operacional. La alternativa seleccionada (MySQL GTID + HAProxy) obtuvo la puntuación más alta (8.75/10) y los resultados confirman que fue la elección correcta.

La arquitectura implementada proporciona bases sólidas para escalamiento futuro. La adición de esclavos adicionales es trivial y el diseño modular facilita la integración de componentes adicionales como caching (Redis), load balancing adicional (más instancias de HAProxy), o incluso migración a Kubernetes para orquestación más sofisticada.

Las lecciones aprendidas durante la implementación, particularmente relacionadas con networking Docker, gestión de streams bash, y configuración de bind addresses, son directamente aplicables a proyectos similares y representan conocimiento valioso para el equipo de desarrollo.

En conclusión, el sistema implementado aborda efectivamente las necesidades de escalabilidad, alta disponibilidad y separación de cargas de trabajo identificadas en la problemática inicial. La solución es robusta, bien documentada, reproducible, y está lista para adaptación a entornos de producción con las mejoras de seguridad y monitoreo discutidas en la sección de trabajo futuro.

En la ejecución práctica se confirmó el funcionamiento completo del sistema en contenedores Docker orquestados con Docker Compose. HAProxy detectó correctamente los tres backends en estado UP, la separación de tráfico fue validada mediante escritura por el puerto 3307 y lectura por el puerto 3308, y los benchmarks con Sysbench permitieron comparar el rendimiento entre cargas read-only y read/write. La prueba read-only alcanzó 534.09 TPS, mientras que la prueba read/write alcanzó 32.51 TPS, evidenciando el beneficio de distribuir las consultas de lectura entre los nodos esclavos.

IX. EVIDENCIAS DE EJECUCIÓN

Las evidencias de ejecución fueron almacenadas en la rama test-funcionando del repositorio de GitHub. En dicha rama se incluyen los scripts de verificación, resultados de Sysbench y archivos generados durante la validación del sistema.

Repositorio: <https://github.com/Nico411/LabHaproxySql/tree/test-funcionando>

Evidencias generadas:

1. docker-compose-ps.txt: estado de los contenedores MySQL y HAProxy.

2. haproxy-stats.csv: estado de backends reportado por el dashboard de HAProxy.
3. verify-replication.txt: validación del estado de replicación en los esclavos.
4. verify-healthchecks.txt: validación de endpoints HTTP de health check.
5. verify-haproxy.txt: validación de frontends, dashboard y backends de HAProxy.
6. sysbench-read-only.log: resultado del benchmark read-only por puerto 3308.
7. sysbench-read-write.log: resultado del benchmark read/write por puerto 3307.
8. resumen-sysbench.txt: resumen comparativo de TPS obtenidos.

Capturas recomendadas para anexar: dashboard de HAProxy con los backends UP, prueba de separación de tráfico por puertos 3307/3308, resultados de Sysbench y evidencia de redetección automática ante caída de un esclavo.

X. REFERENCIAS

- [1] M. Kleppmann, Designing Data-Intensive Applications, O'Reilly Media, 2017, pp. 151-189.
- [2] Oracle Corporation, "MySQL 8.0 Reference Manual - Chapter 17: Replication," MySQL Documentation, 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/replication.html>
- [3] L. Atkins and D. Schwartz, High Performance MySQL, 4th ed., O'Reilly Media, 2021, pp. 423-467.
- [4] W. Tarreau, "HAProxy - The Reliable, High Performance TCP/HTTP Load Balancer," HAProxy Documentation, 2024. [Online]. Available: <http://www.haproxy.org/>
- [5] J. Zawodny and D. Balling, High Performance MySQL, 2nd ed., O'Reilly Media, 2008, pp. 345-398.
- [6] P. Zaitsev, V. Tkachenko, J. Buytaert, C. Tkachenko, and A. Grubshteyn, High Performance MySQL, 3rd ed., O'Reilly Media, 2012, pp. 531-589.